

EvoODE — Projektjournal

Stand: 2026-08-02 · Zeitraum: 2026-04-20 bis 2026-08-02

0. Wie dieses Dokument zu lesen ist

Dieses Journal ist der **rote Faden**: Es erzählt, was gebaut wurde, was verworfen wurde, warum, und was die Zahlen dazu sagen. Es ersetzt keines der bestehenden Dokumente, sondern verbindet sie:

Dokument	Rolle
CLAUDE.md	Zustandsbeschreibung: Architektur, Konventionen, aktuelle Prioritäten
DIARY.md	Chronologisches Protokoll, neueste Einträge zuerst, mit Commit-Hashes
PAPER_1.md	Ausführungsplan für Paper 1, maßgeblich bei Widersprüchen
dieses Journal	Narrativ und Begründungen: der Zusammenhang zwischen den Einträgen

Alle Zahlen stammen aus protokollierten Messungen. Wo eine Aussage eine Vermutung ist, steht das ausdrücklich dabei.

1. Das Projekt in einem Absatz

EvoODE ist ein Julia-Forschungsframework zur datengetriebenen Entdeckung interpretierbarer ODE-Systeme. Der Ansatz unterscheidet sich von SINDy (feste Bibliothek, direkte Regression) und von klassischer genetischer Programmierung (globale Suche aus großen Zufallsstrukturen): EvoODE **beginnt minimal und wächst inkrementell**, und erhöht Komplexität nur, wenn einfachere Strukturen nicht ausreichen. Der wissenschaftliche Kern ist nicht das Fitten von ODEs, sondern die Frage, wie Modellstrukturen aufgebaut, erweitert, validiert und kontrolliert werden sollten.

Forschungsfokus der Dissertation: **effiziente und robuste Suchstrategien für die interpretierbare Entdeckung gekoppelter ODE-Systeme.**

2. Der Zeitstrahl in Phasen

Zeitraum	Phase	Ergebnis
2026-04-20 – 04-22	Fundament	Kernpipeline, EvoGrow v1/v2.1/v2.2, GP-Baseline, Experimentinfrastruktur
2026-04-22 – 05-08	Phase-A-Experiment	300 Läufe, 4,5 Tage Rechenzeit, 300/300 erfolgreich
2026-05-08 – 05-11	Auswertung & Freeze	H1 partial, H2 supported, H3 partial, H4 vakuös
2026-05-17	Strategischer Pivot	Paper 1 wird kein Pretuning-Paper; Gate-Struktur eingeführt
2026-05-30	Gate 1	v2.2 nicht paper-ready → EvoGrow v3 ausgelöst
2026-07-20	v3-Entwurf + Regressionshistorie	Designnotiz, Lockstep-Brücke, append-only Historie
2026-07-22	Kostenzusammenbruch	Volllauf abgebrochen; 62 % der Rechenzeit oberhalb der nötigen Stage
2026-07-22 – 07-23	Screening & Numerik	Screening verworfen, wieder aufgenommen; Numerikbefund; Faktor 6,2
2026-07-29	WP-T2 — die Entscheidungsmessung	Overshoot auf System 26 toleranzinvariant → algorithmisch, v3 bestätigt; Screening ohne Discovery-Gewinn
2026-07-29 – 07-30	v3-Kette scharf geschaltet	v3.3 Kindergenerierung, v3.4 Promotion, v3.5 Metriken; divergenter Pfad erstmals gelaufen
2026-07-30	Zweiter Kostenzusammenbruch	Baseline-v1-Matrix ~2 Wochen, 2/3 Wegwerf-Compute; abgebrochen
2026-07-30	WP-G2.1	Eine vorab festgelegte Do-or-Die-Zelle statt 45 Läufen
2026-07-31	Gate 2	v3 scheitert — verändert nur, <i>wer</i> entscheidet, nicht <i>welche Evidenz</i>
2026-07-31	Look-Ahead im Ableitungsraum	Erst Fehlalarm durch Ableitungsfehler, dann saubere Trennung; <code>r_k</code> als kontaminiert nachgewiesen
2026-07-31 – 08-01	Stage-Cap	Vom Klassifikator zum Suchmechanismus; Sicherheitsprinzip nach zwei Fehlversuchen
2026-08-01	Entscheidungszelle	Overshoot 2 → 0 bei bit-identischem Loss, Struktur unverändert → Symptom statt Ursache
2026-08-02	Bestätigung + Scope-Entscheidung	Overshoot repliziert; „zum Nulltarif“ gilt nicht allgemein; Zweig 1 gewählt
2026-08-02	Phase 3 beginnt	Protokoll-Audit: Zeitgitter passt bei keinem System

3. Die Chronik

3.1 Fundament (2026-04-20 bis 2026-04-22)

Die Kernpipeline `discover()` steht: Daten → Struktur → Parameter → Simulation → Bewertung → Iteration. Implementiert wurden EvoGrow (v1 flach, v2.1 gestuft, v2.2 stage-lokal), eine GP-Baseline, `StagedPolynomialBasis` mit fünf Komplexitätsstufen (linear, selbst-quadratisch, paarweise Kreuzterme, selbst-kubisch, trigonometrisch), `MSELoss` und `BFGSOptimizer`.

Zwei Designachsen wurden bewusst getrennt gehalten und dürfen nicht wieder zusammengelegt werden:

- **Stage Progression Policy** — wann wird eine Stage gehalten, promoviert oder beendet
- **Stage Usage Policy** — wie stark werden neu freigeschaltete Terme danach bevorzugt (:hard , :soft , :passive)

Ebenfalls hier entstanden: `pretune.jl` (Least-Squares-Warmstart über finite Differenzen) und die Experimentinfrastruktur mit Manifest-Generator, sequentiellm Runner und Aggregator, inklusive Per-Run-Dateiprotokoll mit atomaren Writes.

3.2 Das Phase-A-Experiment (2026-04-22 bis 2026-05-08)

`paper1_phaseA_v1`: 10 Systeme × 6 Varianten × 5 Seeds = **300 Läufe**, Laufzeit **4,5 Tage**, Ergebnis 300/300 erfolgreich.

Nebenbefund vom 2026-04-27, der später noch wichtig wird: `profile_init.jl` hing **über 48 Stunden** in einem einzelnen BFGS-Aufruf (Lorenz 3D). Reaktion war die Einführung von `time_limit_s = 300.0` als Wall-Clock-Notbremse im `BFGSoptimizer` — begründet mit „~100× der medianen Aufrufzeit, greift bei normalen Läufen nie“. Diese Annahme erwies sich knapp drei Monate später als falsch (siehe 3.8).

3.3 Auswertung und Freeze (2026-05-08 bis 2026-05-11)

Hypothesenverdikte:

Hypothese	Verdikt	Belegt auf
H1 Stage-Overshoot-Reduktion	PARTIAL	nur System 54 (1 von 6)
H2 kompetitive Rückgewinnungsqualität	SUPPORTED	7 von 8 Systemen
H3 Reduktion verschwendeter Level	PARTIAL	Systeme 11 und 54 (2 von 6)
H4 Ordnung der Usage-Policies	vakuös	alle <code>exact_match = 0</code> , Ordnung nicht unterscheidbar

Das stärkste Ergebnis war ein anderes — EvoGrow gegen die GP-Baseline auf gekoppelten Systemen:

System	Beste EvoGrow	GP	Faktor
31 SIR	7,0e-05	0,314	~4.500×
54 Lorenz	7,4e-04	0,921	~1.200×
26 Lotka-Volterra	2,5e-04	2,98e-03	~12×

Gleichzeitig eine bekannte Schwäche: System 11 ($du = -u^3$) erreicht Loss $\sim 4,4e-15$, aber `exact_match = 0`, weil Wachstum ohne Pruning Nullterme akkumuliert. Als echter algorithmischer Mangel dokumentiert, nicht als Metrikfehler.

Die Phase-A-Ergebnisse sind **eingefroren** und werden für die finalen Paper-Aussagen nicht mehr verwendet.

3.4 Strategischer Pivot (2026-05-17)

Paper 1 wird **kein** Pretuning-Vergleichspaper. Pretuning wird für Paper 1 vollständig deaktiviert (`use_pretuning = false`) und als mögliches Folgepaper zurückgestellt.

Neue Kernfrage: Ist inkrementelles, gestuftes Wachstum ein effektiver Suchmechanismus für interpretierbare ODE-Entdeckung — und wo hilft es, wo versagt es?

H1/H3 „PARTIAL“ wird umgedeutet: nicht als schwaches Ergebnis, sondern als **diagnostisches Signal**, dass systemweites Staging für mehrdimensionale Systeme zu grob sein könnte. Es wird eine Gate-Struktur eingeführt (Gate 1: Ist v2.2 paper-ready? Gate 2: Ist v3 paper-ready?).

3.5 Gate 1 — v2.2 scheitert (2026-05-30)

15 Diagnostikläufe, v2.2 stage_local, use_pretuning = false, n_levels = 30, 3 Seeds je System.

System	Ergebnis	Loss	Stage	Laufzeit
3 Logistic	pruned_match = true, alle Seeds	~7e-10	Overshoot 0–3	—
11 Cubic	pruned_match = true, alle Seeds	~4e-15	4/4	~1,3 s
26 Lotka-Volterra	false, alle Seeds	5e-4 – 1,4e-3	5/3	2.800 – 9.400 s
31 SIR	false, alle Seeds	7e-11 – 1e-4	5/3	—
63 SEIR	false, alle Seeds	9e-4 – 1,8e-3	5/3	11.000 – 31.000 s

Auf System 26 lautet die gefundene Struktur $du_1 = 5,05 \cdot u_1 - 3,87 \cdot u_1 \cdot u_2$, $du_2 = 1,13 \cdot u_1 - 1,80 \cdot u_1^2$ — die Terme sind komplett falsch. Der Loss reißt in Stage 3 auf ~1e-3 auf, und danach verbessert die Eskalation bis Stage 5 nichts mehr; trigonometrische Terme sind hier nutzlos.

Diagnose: Der systemweite Staging-Mechanismus zwingt alle Gleichungen gemeinsam in höhere Stages, sobald der globale Fortschritt stagniert. Das ist kein Metrikfehler, sondern eine echte algorithmische Schwäche.

Entscheidung: v2.2 ist nicht paper-ready. Phase 2 (EvoGrow v3, gleichungsweises Staging) wird ausgelöst.

Nebenbeobachtung, noch offen: Auf System 31 Seed 42 wird bei Loss ~7e-11 ein Spurious-Term $0,0022 \cdot u_1$ nicht weggeprunt, weil die Schwelle $1e-3 \times \max_coeff = 4e-4$ darunter liegt.

3.6 v3-Entwurf und Lockstep-Brücke (2026-07-20)

WP-v3.1 liefert docs/evogrow_v3_design.md: gleichungsweise Stage-Zustände (eq_stages, eq_levels_in_stage, eq_plateau_histories), ein Fortschrittssignal auf Basis des Ableitungsresiduums pro Gleichung, eine gleichungsweise Promotionsregel, gleichungsweise Kindergenerierung und neue Metriken.

WP-v3.2 implementiert EvoGrowV3 bewusst als **Lockstep-Brücke**: Die Variante führt bereits gleichungsweise Zustände, promoviert aber alle Gleichungen gemeinsam und ist dadurch verhaltensgleich zu v2.2. Verifiziert als bit-identisch auf System 3 und 26. Das ist der Regressionsanker, gegen den alle späteren v3-Schritte geprüft werden.

3.7 Regressionshistorie (2026-07-20)

Auf Wunsch entstand eine **longitudinale Historie**, um zu verfolgen, ob Ergebnisse besser oder schlechter werden:

- studies/regression/history.jsonl — append-only, versioniert, ein Record je Zelle

- Schlüssel: (git_hash, variant, system, seed) plus config_fingerprint (SHA256 über die metrikrelevante Konfiguration, erste 16 Hex-Stellen)
- Suite: 5 Systeme (3, 11, 26, 31, 63) × 3 Seeds (42, 123, 7) × 2 Varianten = 30 Zellen

Dazu kam Fortschrittslogging: gestapelte Fortschrittsbalken (ProgressMeter), äußerer Balken über alle Zellen, innerer Balken über die Level eines Laufs, Detailausgabe nur in run.log . Leitlinie war ausdrücklich „so wenig wie möglich, aber so viel wie nötig“.

Ein Rechnerneustart beendete den ersten Volllauf nach 7 von 30 Records. Weil die Historie append-only ist, überlebten die 7. Daraufhin **WP-H1d (Resume)**: Der Runner überspringt beim Neustart alle erfolgreichen Zellen desselben config_fingerprint . Bewusst **nicht** auf git_hash geschlüsselt, damit ältere Records bei unveränderter Konfiguration weiterhin erkannt werden.

3.8 Der Kostenzusammenbruch (2026-07-22)

Der zweite Volllauf wurde nach **23 von 30 Zellen und 40,5 Compute-Stunden abgebrochen** — die Laufzeit war nicht mehr vertretbar.

Befund 1 — der Anker hält, mit einer Ausnahme. v2.2 und v3.2 waren in 7 von 8 überlappenden Zellen bit-identisch. Die Abweichung (System 26 Seed 123: 1,3916e-3 gegen 1,3713e-3) war **kein v3-Bug**, sondern ein Reproduzierbarkeitsleck: BFGSOptimizer wurde ohne time_limit_s konstruiert, der Default von 300 s Wall-Clock wurde an Optim.jl durchgereicht. Damit hing die Zahl der Optimierer-Iterationen von der Maschinenlast ab. Dieselbe Zelle brauchte 13.352 s gegen 20.158 s.

Befund 2 — das Kostenprofil. Rekonstruiert aus run.log über alle 23 Zellen:

Stage	Level	Zeit	Anteil	s/Level
1	124	0,7 h	1,9 %	21
2	129	6,1 h	16,3 %	170
3	94	11,6 h	30,9 %	443
4	55	7,4 h	19,7 %	482
5	48	11,7 h	31,3 %	878

Ein Stage-5-Level kostet das **42-fache** eines Stage-1-Levels. Pro Zelle oberhalb der erwarteten Stage aufsummiert:

24,9 von 40,5 Stunden — 62 % der gesamten Rechenzeit — wurden in Komplexität investiert, die die Systeme nie gebraucht haben.

Auf gekoppelten Systemen liegt der Anteil bei 38–80 %. Extremfall System 63 Seed 123: 8,4 h Laufzeit, davon 6,7 h verschwendet; ein einzelnes Level (16, Stage 4) kostete 3,4 Stunden.

Konsequenz: Laufzeitproblem und Gate-1-Failure-Mode sind **dasselbe Problem**. Die Suche läuft über die nötige Stage hinaus, und jede zusätzliche Stage ist überproportional teurer, weil kubische und trigonometrische Terme steife und divergierende Kandidaten-ODEs erzeugen.

Blocker für Phase B: 63 Systeme × 2 Bedingungen × 3 Seeds = 378 Läufe, davon 240 gekoppelt. Bei ~3,5 h je gekoppelter Zelle sind das **über 800 Stunden ≈ 5 Wochen durchgehend**. Phase B war mit diesem Kostenprofil nicht durchführbar.

3.9 WP-P1 — Determinismus, Budgets, Instrumentierung

Drei Runden (P1, P1b, P1c), jeweils mit Review:

- **Determinismus:** Wall-Clock aus dem Ergebnispfad entfernt (`time_limit_s = 86.400 s` als reine Notbremse, deren Auslösen gezählt wird).
- **Getrennte Budgets** für Screening während der Suche und finale Validierung, mit den heutigen Werten als Default, damit ohne Aktivierung kein Verhalten sich ändert.
- **Instrumentierung** pro Level: Fits, Integrationen, verworfene Integrationen, Zeitanteil Optimierung gegen Simulation, Retcode-Kategorien.

Zwei Reviewbefunde waren substantiell:

1. `isoutofdomain` **war das falsche Primitiv**. Belegt in den DiffEq-Quellen (`OrdinaryDiffEqCore/.../integrator_utils.jl:268-286`): Es verwirft den Schritt und verkleinert `dt`, statt abubrechen — für divergierende Kandidaten also **teurer**. Das Abbruch-Primitiv ist `unstable_check`, dessen Default bereits `any(!isfinite, u)` prüft. Umgestellt.
2. **Der erste Benchmark-Zuschnitt hätte am Problem vorbeigemessen**. Mit 12 Leveln kostet System 26 nur 1,6 min; der Kostenausbruch beginnt bei Level 13 (147 s), 14 (878 s), 16 (611 s), 19 (1.484 s). Budget auf 18 Level korrigiert.

Ergebnis des Mikro-Benchmarks (System 26, Seed 42, 18 Level, je 370 Parameter-Fits):

	A Referenz	B Screening-Budgets	Faktor
Laufzeit	3.222,6 s	1.189,8 s	2,71×
Integrationen	1.741.484	2.488.973	0,70×
Kosten pro Integration	1,763 ms	0,409 ms	4,31×
Solve-Anteil an Laufzeit	95 %	86 %	
Overhead ohne Solve	153 s	166 s	

Sauberster Einzelvergleich: Stage 2, in beiden Fällen exakt 8 Level — A 2.937,9 s, B 297,1 s, **Faktor 9,9**.

Die Zahl, die alles Weitere bestimmt hat: 95 % der Laufzeit stecken in der ODE-Integration, der Overhead außerhalb beträgt 153 s. Ein Screening ohne Integration hätte damit eine Obergrenze von **~21×** ($3.222,6 / 153$).

Zwei Nebentbefunde mit eigener Bedeutung:

- A ist bit-identisch zu Baseline v0 (`0.001391623174905009`) — aber v0 brauchte dafür 30 Level und 3,0 h und lief bis Stage 5, A erreicht denselben Loss in 18 Leveln und 53,7 min bei Stage 3. **Die Level 19–30 kosteten rund 2,1 h und verbesserten nichts**.
- Über alle 23 v0-Zellen: 13 liefen über Level 18 hinaus, und in **allen 13** war der Loss bei Level 18 bereits identisch zum Endergebnis. Das entspricht **15,8 von 40,5 h = 39 %** der Rechenzeit ohne jede Verbesserung.

Eine Kürzung des Level-Budgets wurde trotzdem abgelehnt. Der Loss bliebe identisch, `final_stage`, `stage_overshoot` und `wasted_levels` aber nicht: System 26 Seed 42 zeigt bei 30 Leveln Stage 5 / Overshoot 2, bei 18 Leveln Stage 3 / Overshoot 0. Ein Level-Cap würde das Overshoot-Phänomen **wegschneiden statt messen** — also genau das, was v3 beheben soll.

3.10 Die Screening-Spur (2026-07-22 bis 2026-07-23)

Idee: Die 95 % Solve-Anteil lassen sich nur angreifen, wenn die *Anzahl* der Integrationen sinkt. Kandidaten werden deshalb über ein **Ableitungsresiduum in geschlossener Form** bewertet (die Maschinerie liegt bereits in `pretune.jl`), und nur eine kleine Auswahl wird simuliert.

WP-P2.1 (Designnotiz) legte fest, dass Plateau-Erkennung, Stoplogik und Stage-Promotion weiterhin ausschließlich auf dem **simulierten** Loss arbeiten und der berichtete `loss` ein voll simulierter Loss bleibt. Das Review ergänzte das fehlende Kostenmodell:

Pro Level (20,6 Kandidaten)	Kosten	Faktor
heute: alle per BFGS mit Simulation	170,9 s	1,0×
Top 10 per BFGS	83,0 s	2,1×
Top 10, je 10 Polish-Iterationen	4,2 s	41×
Top 10, je 20 Polish-Iterationen	8,3 s	21×

Daraus die Architektur: **begrenzttes Nachpolieren** statt roher LS-Parameter, weil `plateau_tol = 1e-4` und `loss_tol = 1e-8` auf BFGS-optimierte Losses kalibriert sind.

Der Verlauf war holprig und lehrreich:

1. **WP-P2.2/2b:** Drei Blocker im Review — eine Feldkollision, die Records unbrauchbar gemacht hätte; ein falscher Zähler für die Budget-Erschöpfung; und eine Rangübereinstimmung, die ihre eigene Frage nicht beantworten konnte, weil sie nur über die *ausgewählten* Kandidaten rechnete.
2. **WP-P2.2c (erste echte Zahlen, Toleranz 1e-6):** System 11 funktioniert (rho **+1,0**, 1,53× schneller, korrekte Struktur). System 3 versagt: rho **-0,78**, Loss 3,24e-8 statt 2,66e-10, Stage 5 statt 3, und mit 0,89× sogar **langsamer** als der Referenzpfad.
3. **WP-P2.3 (AIC):** Ergebnis bit-identisch zur Vorrunde, rho unverändert -7/9. Grund, im Nachhinein rechnerisch belegt: Bei $n = 200$ beträgt die AIC-Strafe über $p = 1..6$ höchstens **10 Einheiten**, während der Fit-Term sich schon bei 10 % Residuenunterschied um **19** und bei Faktor 2 um **139** ändert. AIC war hier faktisch eine monotone Transformation des Residuums — und Spearman ist gegen monotone Transformationen invariant. BIC wäre mit $\leq 26,5$ ebenfalls zu schwach gewesen.
4. **Abbruch** per vorab vereinbarter Regel — und wenig später **Wiederaufnahme**, weil die Falsifikation weich war: Die Intervention hatte die Rangfolge gar nicht bewegt, die Hypothese war also ungeprüft, nicht widerlegt.

3.11 WP-T1 — die Numerik-Diagnose (2026-07-22)

Bevor der zweite Screening-Versuch lief, wurde ein Confounder geprüft: Der finale Refit kehrte nach **5 Loss-Auswertungen** mit `retcode = Success` zurück, ohne etwas zu verbessern. Vermutung: Der Bewertungspfad simuliert mit `abstol = reltol = 1e-6` — Gradienten aus finiten Differenzen einer Größe, die nur auf 1e-6 genau berechnet wird, wären in diesem Bereich Rauschen.

Teil A — bestmöglich erreichbarer Loss mit den wahren Parametern:

Toleranz	System 3	System 11
1e-5	2,06e-10	4,79e-13
1e-6	4,40e-12	1,86e-14
1e-8	6,38e-16	1,36e-17
1e-10	4,58e-17	3,84e-19
1e-12	4,34e-17	3,86e-19

Baseline v0 meldet für System 11 einen Loss von 4,402e-15 — besser als das, was die wahren Parameter bei dieser Toleranz erreichen können (1,86e-14). Diese Zahl ist numerisches Rauschen.

Sie steht in Baseline v0, in der Phase-A-Auswertung und in allen bisherigen Regressionsprüfungen.

Teil B — Rauschgrenze des Optimierers. Erreichter Loss aus dem LS-Warmstart:

Toleranz	System 3	System 11
1e-6	3,236e-08	4,606e-15
1e-8	3,236e-08	1,669e-17
1e-10	3,236e-08	4,860e-18
1e-12	3,236e-08	4,856e-18

System 11 bestätigt die Vermutung (der Wert skaliert direkt mit der Toleranz). **System 3 widerlegt sie** — dort ist der Wert über sechs Größenordnungen völlig flach, obwohl der numerische Boden bei 4,40e-12 liegt, also 7.000-fach darunter.

Und damit war die eigentliche Ursache gefunden: 3,236e-08 ist exakt der Wert, bei dem die Screening-Variante hängenblieb. Der Referenzpfad erreicht 2,66e-10 und benutzt **keinen** Warmstart (`use_pretuning = false`). Der ableitungsbasierte Warmstart führt auf System 3 in ein Becken, aus dem BFGS nicht herausfindet — die Screening-Variante hatte also genau das wieder eingeführt, was die Konfiguration bewusst abgeschaltet hatte.

Zwei weitere Befunde nebenbei:

- **Pathologische Line-Search:** Einzelne Fits verbrauchen bei **zwei Parametern** bis zu **39.933 Loss-Auswertungen** (jede eine vollständige Integration) mit Retcode `Failure`, Laufzeiten 3,2 / 10,4 / 23,6 s — bei `maxiters = 200`, also rund 200 Auswertungen pro Iteration. Das erklärt die 7.281 Integrationen pro Fit aus der Regressionsmessung.
- **Sentinel-Loss:** Ein vollständig gescheiterter Fit meldet `final_loss = 1.000e+06` (der Initialwert `l_best`) mit Retcode `Success` und ist von einem echten schlechten Fit nicht unterscheidbar.

3.12 WP-P2.4 — der Durchbruch (2026-07-23)

Zwei getrennte Interventionen, jede einzeln messbar:

1. **Geschachtelter Modellvergleich als Gate** statt additivem Strafterm. Begründung: Kinder sind geschachtelte Obermengen ihrer Eltern, und für geschachtelte Least-Squares-Probleme ist das Residuum monoton nicht-steigend in der Termzahl — ein größeres Modell kann nie schlechter abschneiden. Ein Kind darf seinen Elternteil nur überholen, wenn die Residuenverbesserung größer ist, als ein zusätzlicher Parameter zufällig liefern würde.

2. **Polish-Start vom Screening entkoppelt:** Der LS-Fit dient nur noch der Bewertung, der Startpunkt fürs Nachpolieren kommt vom Referenzpfad.

System 3, Bewertungstoleranz 1e-8:

Bedingung	Zeit	Loss	Stage	Fits	Integrationen
A Referenz	78,6 s	6,25e-9	2	110	275.098
B Residuum + LS-Start (vorher)	353,2 s	3,236e-8	5	241	761.581
C Nested-Gate + LS-Start	49,2 s	3,236e-8	5	241	232.224
D Nested-Gate + entkoppelter Start	12,6 s	2,558e-9	2	61	73.680

D schlägt den Referenzpfad um Faktor 6,2 bei besserem Loss und korrekter Stage.

System 11: alle Bedingungen qualitativ gleichwertig (Loss $\sim 2,0e-16$, Stage 4, `pruned_match true`), D mit 0,6 s gegen 1,1 s am schnellsten.

Die Arbeitsteilung ist sauber getrennt:

- Das **Gate** senkt Kosten und repariert das Ranking: B $\rightarrow$ C bringt 7,2 $\times$ bei identischem Ergebnis, 147 von 400 Kindern scheitern am Gate, die Auswahl weicht in 126 Fällen auf 7 von 20 Leveln ab, und die Rangübereinstimmung kippt von **-0,78 (Median -1,0)** auf **+0,26 (Median +0,48)**.
- Der **entkoppelte Startpunkt** repariert Qualität und Eskalation: C $\rightarrow$ D.

Keine der beiden Interventionen allein hätte gereicht.

Der Nebenfund mit möglicherweise größerer Tragweite: Der **Referenzpfad allein** profitiert schon von der engeren Toleranz.

System 3, Referenzpfad	Zeit	Loss	Stage	Overshoot
bei Toleranz 1e-6	279,3 s	2,66e-10	3	1
bei Toleranz 1e-8	78,6 s	6,25e-9	2	0

Erklärung: Bei 1e-6 erreicht der Optimierer auf Stage 2 die Schwelle `loss_tol = 1e-8` nicht zuverlässig, die Abbruchbedingung feuert nicht, und die Suche eskaliert.

Ein Teil des beobachteten Stage-Overshoots wäre damit ein numerisches Artefakt und keine Eigenschaft der Promotionsregel. Das berührt unmittelbar die 62-Prozent-Rechnung und die Begründung für v3. Bislang: eine Zelle, ein Seed, ein 1D-System — eine Hypothese, kein Befund.

3.13 WP-T2 — die Entscheidungsmessung (2026-07-29)

Die offene Frage aus 3.12 wurde beantwortet. Der Toleranzvergleich lief auf **System 26** — genau dem gekoppelten System, an dem Gate 1 gescheitert ist. Drei Bedingungen, Seed 42, 30 Level: R6 (Referenz, 1e-6), R8 (Referenz, 1e-8), D8 (Screening, 1e-8).

Vor dem Lauf wurde die Prognose **geschärft**: Auf System 3 war der numerische Kanal spezifisch — der Loss operiert nahe `loss_tol = 1e-8`, und bei 1e-6 erreicht der Optimierer die Schwelle nicht, also feuert der Abbruch nicht und die Suche eskaliert. Auf System 26 liegt der Loss-Boden bei $\sim 1,4e-3$, drei

Größenordnungen **über** selbst der 1e-6-Toleranz. `loss_tol` kann dort nie feuern, unabhängig von der Toleranz. **Vorhersage: die engere Toleranz ändert den Overshoot auf System 26 nicht — er ist hier algorithmisch.**

Anker bit-exakt reproduziert: R6 liefert `0.001391623174905009`, `final_stage = 5`, Overshoot 2, `wasted_levels = 8`, `pruned_match = false` — identisch zu Baseline v0. Damit ist die Messung interpretierbar.

Ergebnis — Vorhersage bestätigt: R6 und R8 haben bit-identisch `final_stage = 5`, `stage_overshoot = 2`, `wasted_levels = 8`. Die engere Toleranz senkt nur den Loss ($1,39\text{e-}3 \rightarrow 2,52\text{e-}4$), nicht das Stopp-Verhalten; beide terminieren via `plateau_absolute` und nähern sich `loss_tol` nie.

***Der Overshoot auf System 26 ist algorithmisch, nicht numerisch.** Damit trennt die Messung sauber: auf einfachen Systemen (System 3) numerisch, auf gekoppelten algorithmisch. Das ist eine stärkere Paper-Aussage als „Overshoot ist numerisch“ — und die v3-Begründung ist bestätigt, nicht bedroht.*

Der v3-Beleg steckt in der gefundenen Struktur. Wahrheit: $du_1 = 3 \cdot u_1 - u_1^2 - 2 \cdot u_1 \cdot u_2 \mid du_2 = 2 \cdot u_2 - u_1 \cdot u_2 - u_2^2$. R8 nach Pruning trifft `du1` **exakt** ($3 \cdot 0.03 \cdot u_1 - 1 \cdot 07 \cdot u_1^2 - 1 \cdot 99 \cdot u_1 \cdot u_2$), aber `du2 = {u1, u1^2}` ist **komplett falsch**. Eine Gleichung gelöst, die andere im Blindflug — und der globale Plateau-Mechanismus eskaliert Stage 4/5 für **beide**, obwohl Gleichung 1 längst fertig ist. Das ist exakt die Signatur, die v3 (gleichungsweise Promotion) auflösen soll: gelöste Gleichung einfrieren, nur die offene weiterwachsen.

Screening auf gekoppeltem System: schnell, aber kein Discovery-Gewinn. D8 gegen R8 (beide 1e-8): D8 braucht **98.253 Integrationen gegen 3.348.287** (34× weniger), aber beide `pruned_match = false`, beide Overshoot 2. Zwei Befunde entwerfen den Geschwindigkeitsvorteil als Discovery-Hebel:

- **Ranking-Kollaps:** `rank_agreement_spearman` Median **-0,014**. Das FD-Ableitungs-Screening rankt Kandidaten auf dem gekoppelten System praktisch nicht wie der echte Loss. Der 34×-Vorteil kommt aus *wenig integrieren*, nicht aus gutem Diskriminieren.
- **Nested-Gate inert:** `selection_diff_from_residual = 0` über alle Level. Der WP-P2.4-Durchbruch, der auf System 3 das Ranking reparierte, ändert auf System 26 keine einzige Auswahl. Der Gate-Nutzen ist systemabhängig.

Fazit: Screening ist eine **Performance-Optimierung**, kein Discovery-Qualitäts-Hebel. Es gehört als optionale Beschleunigung dokumentiert, nicht in den Kern-Claim; der Ranking-Kollaps muss in die Discussion.

Messvorbehalt — die Wall-Clock ist kontaminiert. Der PC wurde während des Laufs 2× zugeklappt (Arbeitsweg, je ~45 min) und es lief Nebenlast. Das verfälscht **ausschließlich** die Zeitachse (`elapsed_s`, `s/Level`, der zitierte ~6×-Speedup) — diese Zahlen dürfen nicht als präzise Messwerte gelten. Die tragenden Schlüsse stehen trotzdem, weil sie auf **Zählungen** ruhen, nicht auf Zeiten: der Anker ist bit-exakt (Determinismus ungebrochen), `time_limit_s` war nie bindend ($\text{max } 31.413 \text{ s} \ll 86.400 \text{ s}$, keine Iteration abgeschnitten), und Stage/Overshoot/Struktur/Integrationszählungen sind maschinenlastunabhängig. Konsequenz fürs Paper: **Kosten über Integrationszählungen berichten, nicht über Wall-Clock.**

Eine eigene Korrektur gehört hierher: Die früher genannten „62 % Overshoot-Kosten“ waren die *Wall-Clock*-Sicht. Suspend-fest und deterministisch sind es auf System 26 **8 von 25 Leveln jenseits der erwarteten Stage 3, ~25 % der Integrationen** (Stages 4+5: 832.350 von 3.348.287 Solves). Die Differenz

zur Wall-Clock-Zahl kommt daher, dass späte Integrationen einzeln teurer sind — was teils echte Numerik ist, aber eben auf der kontaminierten Achse liegt.

3.14 EvoGrow v3 wird scharf geschaltet (2026-07-29 bis 2026-07-30)

Mit der bestätigten Begründung wurde die v3-Kette in regressionssicheren Schritten umgesetzt. Jeder Schritt wurde **statisch** geprüft (Julia nicht gestartet, um die zu dem Zeitpunkt noch laufende Baseline nicht um CPU zu bringen); die Ausführungs-Evidenz liefert jeweils der Test-/Smoke-Lauf.

- **WP-v3.3 — gleichungsweise Kindergenerierung** (`src/structure/evogrow_v3_childgen.jl`). Neu freigeschaltete Terme werden pro Gleichung anhand ihres eigenen Stage-Zustands zugelassen; echte Kreuzterme über die Paarregel `min(eq_stages[v] for v in vars(t)) >= stage(t)`. Entscheidend: bei **uniformen** Stages — dem Lockstep-Zustand — delegiert der Pfad an den unveränderten v2.2-Code mit identischer RNG-Reihenfolge, ist also **strukturell bit-identisch**. Der Pro-Gleichungs-Pfad ist bis v3.4 im echten Lauf tot.
- **WP-v3.4 — gleichungsweise Promotionsregel** (`src/structure/evogrow_v3_promote.jl`). Das Fortschrittssignal ist das **Ableitungsresiduum pro Gleichung** `r_k`, ausgewertet auf der beobachteten Trajektorie (`estimate_derivatives` aus `pretune.jl`, keine Integration im Normalpfad, RNG-neutral). Promotion pro Gleichung an drei Bedingungen (Budget, `r_k`-Plateau, `r_k > loss_tol`) plus Maxstufen-Guard; die globale Termination wird **vor** der Promotion geprüft, damit der `loss_tol`-Stopp Vorrang behält. Das ist der Schritt, der die v2.2-Gleichheit **bewusst bricht** — der Bit-Identitäts-Smoke wurde durch einen Scalar-Promote-Smoke ersetzt, und die Verifikation ist deterministische Unit-Test-Logik statt Bit-Gleichheit (alle vier Promotionsbedingungen einzeln, plus der `r_k`-Signaltest).
- **WP-v3.5 — Pro-Gleichungs-Metriken + Integrationsnachweis** (`eq_overshoot`, `eq_wasted_levels`). Rein abgeleitete Metriken, in den Regressions-Record aufgenommen (`nothing` bei Nicht-v3-Varianten, wie `eq_final_stages`), ohne `config_fingerprint`- oder Verhaltensänderung. Der zweite Zweck war das Schließen einer **Integrationslücke**: der divergente Pfad — v3.3-Kindgenerierung und v3.4-Promotion im Zusammenspiel — war bis dahin nur mit injizierten Stages unit-getestet, **nie end-to-end in einem echten gekoppelten Lauf**. Ein billiger synthetischer 2D-Smoke (eine Gleichung exakt linear → bleibt Stage 1, die andere braucht Stage 2 → steigt) lässt die Stages erstmals real divergieren und testet die neuen Metriken zugleich.

Damit ist die v3-Kette **code-seitig komplett** (v3.2 Brücke → v3.3 → v3.4 → v3.5). Was aussteht, ist die wissenschaftliche Validierung (WP-v3.6): zeigen, dass v3 den Overshoot auf den gekoppelten Systemen 26/31/63 gegenüber v2.2 senkt. Diese Validierung braucht Vergleichsdaten — und genau daran hakt es (3.15).

3.15 Der zweite Kostenzusammenbruch — die Baseline-v1-Matrix (2026-07-30)

Für die Validierung wurde eine neue Regressionsbaseline gestartet (nötig, weil sich die Konfiguration seit Baseline v0 geändert hat — `time_limit_s` ist jetzt explizit, der Fingerprint umfasst neue Felder). Nach **~24 Stunden** war der Lauf bei **6 von 45 Läufen (~13 %)** und hing im ersten gekoppelten System-26-Lauf, der allein den Großteil des Tages fraß. Der Lauf wurde abgebrochen. Die Diagnose deckte ein Design- und ein Scope-Problem auf:

Der Runner ist zu groß. Die Matrix ist **3 Varianten** (`evogrow_v2_2_stage_local`, `evogrow_screening_derivative`, `evogrow_v3`) × 5 Systeme × 3 Seeds = **45 Läufe**. Bei `USE_PRETUNING = false` und `N_LEVELS = 30` kostet ein einzelner gekoppelter Lauf ~16 h (WP-T2-Anker). Über alle

gekoppelten Zellen dreier Varianten summiert sich das auf **~1,5–2 Wochen** durchgehend.

Und ein großer Teil davon ist wertlos:

- Die **evogrow_v3 -Spalte lief mit v3.3-Code** — der Julia-Prozess hatte den Code beim Start geladen, vor v3.4/v3.5. Diese Spalte ist damit Lockstep ($\equiv$ v2.2) und **kein** Beleg für das echte v3.4-Verhalten. Die 9 teuren gekoppelten v3-Läufe müssen für WP-v3.6 mit aktuellem Code ohnehin neu gefahren werden.
- Die **screening -Spalte** ist als Kern-Claim erledigt (3.13) und fürs Paper irrelevant.

Was tatsächlich gebraucht wird, ist genau **eine** Sache: die **v2.2-Referenz** unter dem neuen Fingerprint (15 Läufe, davon 9 teure gekoppelte).

Ein Nebenfund korrigiert eine frühere Überzeichnung: `BFGS_TIME_LIMIT_S = 86.400` (24 h pro Fit) ist zwar unangemessen für einen 45-Lauf-Runner, aber **nicht** der Kostentreiber — WP-T2 zeigt, dass kein Fit dem Limit nahekam. Die ~ 16 h je gekoppelter Lauf entstehen durch die *Anzahl* teurer Fits (`pretuning = false`, 30 Level, steife Kandidaten-ODEs), nicht durch einen hängenden Fit. Das Zeitlimit zu senken ist Hygiene, kein Speedup. Der einzige echte Hebel im Scope ist die **Varianten-Kürzung** (45 $\rightarrow$ 15).

Bewusst offen gelassen: ob die v2.2-Referenz „nur“ durch Varianten-Kürzung ($\sim$ Tage, sicher) oder zusätzlich durch ein Überdenken von `N_LEVELS / pretuning` für die Regression billiger werden soll. Letzteres ändert, was gemessen wird, und wurde als Entscheidung mit klarem Kopf vertagt — nicht im Reaktionsmodus. Diese Entscheidung ist der aktuelle Engpass (siehe 7.1).

3.16 WP-G2.1 — eine Zelle statt einer Matrix (2026-07-30)

Aus dem Scope-Problem von 3.15 wurde die Konsequenz gezogen, die Evidenz zu verkleinern statt sie zu verbilligen. Der v2.2-Arm war bereits eingefroren und von WP-T2 bit-exakt reproduziert — es genügte also **ein** frischer v3-Lauf, um zu entscheiden. Statt 45 Läufen: **System 26, Seed 42**, vorab festgelegt als Do-or-Die-Zelle.

Umgesetzt wurden ein Zwei-Varianten-Runner, ein Einzelzell-Selektor über Umgebungsvariablen, die Senkung von `BFGS_TIME_LIMIT_S` von 24 h auf 1.800 s (reine Notbremse, greift nie) und ein Readout mit **vorab festgelegten** Kriterien: `du1` bleibt auf Stage 3, `du1`-Support exakt $\{u_1, u_1^2, u_1 \cdot u_2\}$, Loss nicht schlechter als der Anker. Vorab festgelegt, damit hinterher nicht die Kriterien an das Ergebnis angepasst werden können.

3.17 Gate 2 — v3 scheitert (2026-07-31)

Der Lauf ergab `eq_final_stages = [5, 5]`, `eq_overshoot = [2, 2]`, und `du1` erhielt zusätzlich einen Fremdterm `u2`. Von den drei Kriterien war nur das dritte erfüllt — und zwar deutlich: der Loss lag mit $2,52e-4$ um Faktor 5,5 **besser** als der v2.2-Anker. **Die Fitqualität war nie das Problem; gescheitert ist die Komplexitätsallokation, also genau das, wofür v3 gebaut wurde.**

Die Diagnose ist der eigentliche Ertrag. Die v3-Promotionsregel fragt pro Gleichung: genug Budget, `r_k` plateau, `r_k > loss_tol = 1e-8`. Die dritte Bedingung ist auf gekoppelten Systemen unerreichbar — der Fehlerboden liegt bei $1e-3$ bis $1e-4$. Für eine bereits korrekt modellierte Gleichung sind damit dauerhaft alle drei Bedingungen erfüllt, und die Regel kann **Untermodellierung** nicht von einem **irreduziblen Fehlerboden** unterscheiden. Beide sehen identisch aus: flaches Residuum oberhalb von $1e-8$.

v3 hat verändert, wer entscheidet, aber nicht, welche Evidenz eine Promotion rechtfertigt.

Das ist ein sauberes, publizierbares Negativergebnis.

3.18 Das Stage-Zünd-Problem und der billige Look-Ahead (2026-07-31)

Aus dem Scheitern entstand die schärfere Problemformulierung. Ein rein *relatives* Kriterium statt des absoluten Zielwerts löst es nämlich ebenfalls nicht. Zwei Gegenbeispiele zeigen den Konflikt:

- **Lotka-Volterra** (wahre Stage 3): Best-Loss $3,84\text{e-}2 \rightarrow 3,35\text{e-}3 \rightarrow 1,39\text{e-}3 \rightarrow \text{flach} \rightarrow \text{flach}$. Richtig wäre: nach Stage 3 **stoppen**.
- **$du = -u^3$** (wahre Stage 4): $2,96\text{e-}1 \rightarrow 3,43\text{e-}3 \rightarrow \text{flach} \rightarrow \mathbf{4,40\text{e-}15}$. Richtig wäre: über die flache Stage 3 hinweg **weitergehen** — die in 1D sogar leer ist.

Beide Entscheidungsmomente sehen lokal gleich aus: Loss um $1\text{e-}3$, aktuelle Stage bringt nichts, Verlauf flach. Der Unterschied liegt ausschließlich in Information über *zukünftige* Termgruppen. **Das Stage-Zünd-Problem ist damit ein Look-Ahead-Problem unter Unsicherheit.** Ein vollständiger simulationsbasierter Look-Ahead ist wegen der ODE-Kosten untragbar — also die Frage, ob ein billiger Test im *Ableitungsraum* trägt: Besitzt die volle Termbibliothek einer künftigen Stage überhaupt zusätzliche Erklärungskraft für die numerisch geschätzten Ableitungen, auf zurückgehaltenen Zeitpunkten?

WP-L1 — der Test scheint zu scheitern, und das ist ein Messartefakt. Die erste Probe über zehn Benchmark-Systeme meldete: keine Trennung. Die Rauschboden-Zeilen, die in der Spec verlangt worden waren, zeigten warum. System 26 du_2 ist bei Stage 3 exakt darstellbar, das Holdout-Residuum der vollen Stage-3-Bibliothek müsste also auf dem analytischen Boden von $4,3\text{e-}11$ liegen — es lag bei $3,7\text{e-}3$, **acht Größenordnungen darüber**. Und auf System 11 hatte das analytisch wahre RHS ein Residuum von 4,606, während der LS-Fit derselben wahren Struktur auf 1,159 kam: **der Fit schlägt die Wahrheit um Faktor 4**, was nur beim Fitten von Rauschen möglich ist. Ursache war `estimate_derivatives` — ein einfacher zentraler Differenzenquotient, dessen Fehler im Transienten von Ordnung 1 ist, also größer als das gesuchte Signal. Das ist dieselbe Signatur wie bei WP-T1.

WP-L2 — mit brauchbarer Ableitung trennt der Test sauber. Ein lokal-polynomialer Glätter senkt den mittleren Ableitungsfehler um Faktor 10; höhere FD-Ordnung bringt nur $1,4\times$ — **Glättung ist der Hebel, nicht die Ordnung**. Damit:

Stage	System 26 du_2 (wahr: 3)	System 11 du_1 (wahr: 4)
2	$3,34\text{e-}6$	$4,53\text{e-}5$
3	$5,24\text{e-}13$	$4,53\text{e-}5$ (leer, identisch)
4	$2,70\text{e-}11$ (schlechter)	$7,60\text{e-}12$
5	$1,81\text{e-}11$	$5,05\text{e-}9$ (schlechter)

Beide Klippen sitzen exakt auf der wahren Stage, danach wird es schlechter. Die Kernfrage ist damit beantwortet.

Und ein Nebenbefund mit größerer Tragweite: `r_k`, das v3-Promotionssignal, ist dasselbe Ableitungsresiduum. Auf System 26 liegt der Boden mit *wahrer* Struktur und *wahren* Parametern bei $[1,808; 0,520]$, das gefittete volle Stage-3- `r_k` dagegen bei $[0,142; 0,0394]$ — der Boden liegt um **Faktor 13 darüber**. `r_k` misst also nicht strukturelle Angemessenheit, sondern wie viel Ableitungsfehler ein Modell absorbieren kann, und diese Kapazität wächst mit der Termzahl. Das Signal ist systematisch nach „mehr Terme helfen“ verzerrt. **v3s Scheitern hat damit eine zweite, numerische Ursache zusätzlich zur Evidenz-Diagnose.**

WP-L3 — Regel und Grenzen vermessen. Ein Boden-Gate (ein Gewinn zählt nur oberhalb des Rauschbodens) bringt die Konfusionsmatrix über die 16 exakten Gleichungen auf 12 exakt / 0 Overshoot / 4 Undershoot. Ein Dichte-Sweep trennt zwei bis dahin konfundierte Erklärungen: auf System 54 ist die Stage-3-Klippe **sampling-begrenzt** und erscheint ab doppelter Dichte; auf System 63 bleibt das Rangdefizit bei *jeder* Dichte bestehen — es ist die Erhaltungsgröße (die SEIR-Zustände summieren sich zu einer Konstanten).

3.19 Der Stage-Cap — vom Klassifikator zum Mechanismus (2026-07-31 bis 2026-08-01)

Der entscheidende Designbefund kam beim Lesen: **das Gate ist suchunabhängig.** Es hängt nur an Trajektorie, Basis, Gleichung und Stage — nicht an der Population. Damit braucht es weder spekulatives Unlock noch Checkpoint noch Rollback: `max_useful_stage_k` wird einmal vor der Suche berechnet und wirkt als Obergrenze. Das ersetzte eine ganze Maschinerie durch eine Vorberechnung.

Der Weg dahin war die instruktivste Fehlerkette des Projekts:

- **WP-L4** lieferte den Cap, aber mit einem Sicherheitsdefekt: System 63 bekam `[1,1,1,1]` gegen die wahre Struktur `[3,3,1,1]`. Für `du1 / du2` wäre die Wahrheit damit **strukturell unerreichbar** gewesen. Ursache: „Residuum schon bei Stage 1 unter dem Boden“ wurde als „Stage 1 genügt“ gelesen statt als „nicht beurteilbar“.
- **WP-L5** sollte das beheben und machte es schlimmer — Verletzungen von 2 auf 4, und kein System behielt einen nützlichen Cap. **Die Ursache lag in der Spec, nicht in der Umsetzung:** die Vorgabe „Residuum unter dem Boden → nicht beurteilbar → kein Cap“ ist falsch, denn **den Boden zu erreichen ist genau das, was ein korrektes Modell tut.** Die Regel konnte damit auf keinem lösbaren System je einen Cap setzen. Zweiter, unabhängiger Defekt: der Look-Ahead-Horizont war auf 1 geschrumpft und konnte keine nutzlose Zwischenstufe mehr überbrücken — genau das Gegenbeispiel aus 3.18, wiedereingeführt.
- **WP-L5b/L5c** reparierten beides: unterschieden wird jetzt, ob das Residuum auf dieser Stufe *auf* den Boden **fiel** (positive Evidenz, hier cappen) oder schon *vorher* dort lag (keine Information, kein Cap); der Horizont umfasst zwei *anwendbare* Stufen, leere verbrauchen keinen.

Daraus das leitende Prinzip, das die Asymmetrie der Kosten spiegelt — ein falscher Cap macht die Wahrheit unerreichbar, ein fehlender kostet nur den Status quo:

Ein Cap darf nur auf positive Evidenz gesetzt werden, nicht auf die Abwesenheit von Evidenz.

Ergebnis nach WP-L5d, unabhängig nachgerechnet: $3 \rightarrow [2]$, $11 \rightarrow [4]$, $26 \rightarrow [3,3]$, $31 \rightarrow [3,3]$, $63 \rightarrow$ alle ungecappt, $54 \rightarrow [\text{ungecappt}, 2, 2]$. Suiteweit **2 Verletzungen, 8 von 16 Gleichungen gecappt, 18 Stufen gespart** — die Sicherheit ist also nicht durch Nichtstun erkaufte. Die zwei Verletzungen sind die aus WP-L3 bekannte Auflösungsgrenze auf Lorenz. **Damit hat die Regel eine geschlossene Charakterisierung: sicher, wo die Ableitungsschätzung die Struktur auflöst, unsicher genau dort, wo sie es nicht tut — und das ist vorab am Rauschboden ablesbar.**

3.20 Die Entscheidungszelle — Symptom statt Ursache (2026-08-01 bis 2026-08-02)

Der gecappte Lauf auf 26/42 lieferte das schärfste Ergebnis des Projekts, und zwar in zwei entgegengesetzte Richtungen.

	v2.2-Anker	v3 Gate 2	gecappt
Loss	1,3916e-3	2,5196e-4	2,5196e-4
eq_overshoot	2	[2, 2]	[0, 0]
eq_wasted_levels	8	—	[0, 0]
du2-Support	{u1, u1 ² }	—	{u1, u1²}

Die Komplexitätsallokation ist gelöst. Overshoot $2 \rightarrow 0$, verschwendete Level $8 \rightarrow 0$ — und der Loss ist **bit-identisch** zum ungecappten v3-Lauf. Die Stufen 4 und 5 haben dort also buchstäblich nichts beigetragen; die Eskalation war reine Verschwendung.

Die strukturelle Wiederfindung ist unverändert. du2 ist exakt das, was v2.2 drei Methodengenerationen früher fand, obwohl die Wahrheit $2 \cdot u_2 - u_1 \cdot u_2 - u_2^2$ die ganze Zeit auf Stage 3 und damit *innerhalb* des Caps bereitlag.

Die Stage-Eskalation war ein Symptom, nicht die Ursache. Die Suche auf die richtige Stufe zu zwingen verbessert die Entdeckung nicht um einen einzigen Term.

Die Bestätigung relativiert die Kostenaussage. Vier weitere Zellen (26/123, 31/42, 31/123, 31/7) bestätigen den Overshoot-Effekt — alle vier auf $[0, 0]$. Aber „zum Nulltarif“ gilt nur für System 26: auf System 31 ist der gecappte Lauf 50-fach schlechter (Seed 123) und bei Seed 42 um acht Größenordnungen ($1,7e-2$ gegen $6,8e-11$). Auf System 31 senken die Stufen 4/5 den Loss real, wenn auch mit falschen Termen. **Der Cap tauscht Fitqualität gegen Komplexitätsdisziplin, und ob der Tausch gratis ist, hängt am System.**

Der Vergleich ist zudem **konfundiert**: die gecappte Variante ist v3 *plus* Cap, und für System 31 existiert kein v3-Record. Eine Zelle erlaubt die Trennung aber schon jetzt — auf 31/42 ist der Cap **nicht bindend** (v2.2 endete dort von selbst auf Stage 3), und trotzdem bricht der Loss ein. Der Einbruch kann folglich nicht vom Cap kommen, sondern muss aus dem v3-Unterbau stammen: dem r_k -Signal, das in 3.18 als kontaminiert nachgewiesen wurde. Drei ungecappte v3-Läufe auf System 31 stehen aus.

Die Scope-Entscheidung fiel daraufhin auf **Zweig 1, angereichert**: die finale Variante trägt den Cap, das Paper wird die mechanistische Claim-C-Studie, und die Kette $v2.2 \rightarrow v3 \rightarrow$ gecappt wird als dokumentierte Failure-Analyse mit einem quantifizierten Positivergebnis geführt. Das Paper um den Look-Ahead herum neu zu bauen wäre schwächer belegt, weil der Mechanismus genau das Versagen nicht behebt, das Phase 2 ausgelöst hat.

3.21 Phase 3 beginnt — das Protokoll-Audit (2026-08-02)

Für den Vergleich mit publizierten ODEBench-Zahlen wurde das Audit begonnen. Die EvoODE-Seite ist gegen den Datensatz verifiziert, die Spalten der publizierten Quellen sind ausdrücklich als **ungeprüft** markiert.

Anfangsbedingungen passen: alle zehn u_0 reproduzieren exakt den *ersten* der beiden Anfangsbedingungssätze des Datensatzes. **Das Zeitgitter passt bei keinem einzigen System:** der Datensatz liefert durchgängig 512 Punkte über $t \in [0, 10]$, EvoODE nutzt pro System eigene Fenster mit 10 bis 20 Punkten pro Zeiteinheit — 2,6- bis 5,1-fach dünner, teils mit deutlich längeren Horizonten (System 63 bis $t = 30$).

Die Querverbindung macht das mehr als eine Formalie: das Datensatz-Gitter ist auf System 54 **2,56-fach dichter** als unseres — genau der Bereich, ab dem WP-L3 die Stage-3-Klippe als auflösbar gemessen hat. Der Wechsel auf das Datensatz-Gitter würde also plausibel eine der beiden verbliebenen Cap-Verletzungen beseitigen. Das ist eine Vorhersage aus gemessenem Verhalten, kein Ergebnis.

Daraus eine Entscheidung, die **vor** der Phase-B-Generierung fallen muss, weil sie die Trajektorien und damit alles Nachgelagerte bestimmt: Datensatz-Gitter übernehmen (Vergleichbarkeit plausibel, doppelte Laufzahl durch zwei IC-Sätze, kein bestehendes Ergebnis trägt über) oder beim eigenen Gitter bleiben (bestehende Ergebnisse gelten, publizierte Zahlen bleiben rein kontextuell — und das muss im Paper konsistent so stehen).

Parallel läuft die Klassifikation aller 63 Systeme in exakt/surrogat (117 Gleichungen; Operatoren ****** 42×, **sin** 16, **cos** 8, dazu **exp**, **log**, **cot**, **Abs** — plus additive Konstanten, für die die Basis gar keinen Term hat). Diese Zahl ist selbst ein Ergebnis: die Menge der exakten Systeme bestimmt, auf wie vielen Systemen Paper 1 überhaupt strukturelle Wiederfindung berichten kann.

4. Was implementiert ist und funktioniert

Komponente	Status
<code>discover()</code> Kernpipeline	stabil seit 2026-04-20
EvoGrow v1, v2.1, v2.2	stabil
EvoGrowV3 (gleichungsweise)	vollständig; an Gate 2 gescheitert , Promotionssignal <code>r_k</code> als kontaminiert nachgewiesen
EvoGrowStageCapped	Look-Ahead-Cap als Suchvariante; Caps auf 4 von 5 Regressionssystemen korrekt
<code>stage_cap.jl</code>	datenbasierte Cap-Berechnung, kein Ground-Truth-Kanal; Aggregationsmodi + Horizont
EvoGrowScreening	implementiert; nur Performance-Hebel, kein Discovery-Gewinn (WP-T2)
GPStructureSearch	Vergleichsbaseline
StagedPolynomialBasis, PolynomialBasis	stabil
BFGSOptimizer	+ deterministische Budgets, frühe Verwerfung, Retcode-Kategorien
Experimentinfrastruktur	Manifest, Runner, Aggregator, atomare Writes
Regressionshistorie	append-only, Fingerprint-basiert, Einzelzell-Selektor
Look-Ahead-Diagnostik	<code>studies/lookahead/</code> : Stage-Potential-Probe, Schätzervergleich, Boden-Gate + Dichte-Sweep
Gate-2-Readout	vorab festgelegte Kriterien, Record-Auswahl fingerprint-unabhängig
Protokoll-Audit	<code>docs/paper1_odebench_protocol_alignment.md</code> , EvoODE-Seite verifiziert

5. Was verworfen wurde — und warum

Verworfen	Grund	Beleg
Pretuning als Paper-1-Thema	Strategischer Fokuswechsel auf gestuftes Wachstum	2026-05-17
v2.2 als finale Paper-Variante	Systemweites Staging versagt auf gekoppelten Systemen	Gate 1
<code>isoutofdomain</code> für frühe Verwerfung	Verkleinert <code>dt</code> , bricht nicht ab — teurer statt billiger	DiffEq-Quellen
Level-Budget-Kürzung auf 18	Loss identisch, aber Overshoot-Metriken verfälscht	Stage 5/Over 2 bei 30 gegen Stage 3/Over 0 bei 18
AIC und BIC als Komplexitätsstrafe	Bei $n = 200$ vom Fit-Term dominiert	WP-P2.3, Ergebnisse bit-identisch
LS-Warmstart als Polish-Startpunkt	Führt auf System 3 in ein Becken bei 3,236e-08	WP-T1
Screening als Kern-Claim	Ranking-Kollaps (Spearman-Median $-0,014$), Nested-Gate inert	WP-T2
Baseline-v1-Matrix mit 3 Varianten	2/3 Wegwerf-Compute, ~ 2 Wochen	2026-07-30, abgebrochen bei 6/45
v3 als finale Paper-Variante	Keine Entkopplung; verändert nur, <i>wer</i> entscheidet, nicht <i>welche Evidenz</i>	Gate 2, <code>eq_overshoot</code> = $[2, 2]$
Spekulatives Unlock mit Checkpoint/Rollback	Unnötig — das Gate ist suchunabhängig und einmal vorberechenbar	3.19
Höhere FD-Ordnung als Ableitungs-Fix	1,4× gegen 10× durch Glättung	WP-L2
„Residuum unter dem Boden → kein Cap“	Den Boden zu erreichen ist, was ein korrektes Modell tut	WP-L5, Verletzungen $2 \rightarrow 4$
Paper um den Look-Ahead herum	Löst die Allokation, nicht die Wiederfindung	3.20

6. Die wichtigsten Zahlen an einem Ort

Kostenprofil (23 Zellen, 40,5 h): Stage 1 $\rightarrow$ 21 s/Level, Stage 5 $\rightarrow$ 878 s/Level (Faktor 42). 62 % der Rechenzeit oberhalb der nötigen Stage.

Gate 2 (System 26, Seed 42): v3 `eq_final_stages` = $[5, 5]$, `eq_overshoot` = $[2, 2]$, Loss $2,52e-4$ — Faktor 5,5 besser als der v2.2-Anker. Fitqualität war nie das Problem.

Ableitungsschätzung (WP-L2): mittlerer RMS-Fehler `central` $1,75e-2$, `fd4` $1,24e-2$, `local_poly` $1,78e-3$. Glättung bringt 10×, höhere Ordnung 1,4×.

Stage-Potential nach Korrektur der Ableitung: System 26 `du2` fällt bei Stage 3 auf $5,24e-13$ (vorher $3,7e-3$ — zehn Größenordnungen), System 11 bei Stage 4 auf $7,60e-12$; beide verschlechtern sich danach.

`r_k`-Kontamination (System 26): Boden mit wahrer Struktur und wahren Parametern $[1,808; 0,520]$ gegen gefittetes Stage-3-`r_k` $[0,142; 0,0394]$ — Boden **Faktor 13 darüber**.

Stage-Cap, suiteweit: 2 Verletzungen, 8 von 16 Gleichungen gecappt, 18 Stufen gespart. Caps: 3 → [2], 11 → [4], 26 → [3,3], 31 → [3,3], 63 → ungecappt, 54 → [uncappt, 2, 2].

Entscheidungszelle (26/42, gecappt): Overshoot 2 → 0, wasted 8 → 0, Loss bit-identisch zu ungecapptem v3, Parameter-Fits 390 gegen 530. du2 -Support unverändert gegenüber v2.2.

Bestätigung (4 Zellen): Overshoot überall [0,0]. Loss gegen v2.2: 26/123 gleichauf, 31/7 leicht schlechter, 31/123 50-fach schlechter, **31/42 acht Größenordnungen schlechter**.

Protokoll: Datensatz 512 Punkte über $t \in [0,10]$ für alle Systeme; EvoODE 10–20 Punkte pro Zeiteinheit bei eigenen Fenstern. Kein System trifft das Gitter.

7. Offene Punkte, nach Dringlichkeit

1. **v3 ungecappt auf System 31 (Seeds 42/123/7).** Trennt Cap-Effekt von v3-Effekt. Ohne das ist „der Cap kostet Fitqualität“ nicht belegbar — es könnte vollständig ein v3-Effekt sein. Drei externe Läufe.
2. **Gitter-Entscheidung vor Phase B.** Datensatz-Gitter übernehmen oder eigenes behalten. Bestimmt alle Trajektorien und damit alles Nachgelagerte; die Systemklassifikation ist davon unabhängig.
3. **Systemklassifikation aller 63 Systeme** (WP-P3.1, läuft). Die Zahl der exakten Systeme bestimmt, worüber das Paper strukturelle Wiederfindung berichten kann.
4. **Publizierte Protokolle einlesen.** Ohne sie bleiben externe Zahlen kontextuell; die vier offenen Fragen stehen im Audit-Dokument.
5. **Zwei Cap-Verletzungen auf System 54** — verstanden als Auflösungsgrenze, möglicherweise durch die Gitter-Entscheidung von selbst erledigt.
6. **System-31-Fix ohne belegte Ursache.** Die Diagnose lief ins Timeout, die Reparatur erfolgte per Schlussfolgerung. Erste Stelle zum Nachsehen, falls die Regel überrascht.
7. **Pathologische Line-Search** (bis 39.933 Auswertungen bei zwei Parametern) und **Sentinel-Loss 1e6 mit Retcode Success** — unangetastete Kosten- und Robustheitshebel.
8. **Pruning-Schwelle** $1e-3 \times \max_coeff$ möglicherweise zu streng.

Ausdrücklich außerhalb von Paper 1: die Suchkraft *innerhalb* einer Stufe — Populationsgröße, Kindergenerierung, Parsimonie-Druck. Das ist die eigentliche offene Forschungsfrage nach 3.20, aber eine eigene Linie.

8. Der rote Faden

Das Projekt hat eine Frage über drei Methodengenerationen verfolgt und sie beantwortet — nur anders, als erwartet.

Die Ausgangsthese war, dass gestuftes Wachstum die Komplexität kontrolliert und dass das Versagen auf gekoppelten Systemen an der *systemweiten* Stufenlogik liegt. Gate 1 belegte das Versagen, v3 dezentralisierte die Entscheidung — und Gate 2 zeigte, dass Dezentralisierung nichts bringt, weil beide lokalen Automaten dieselbe untaugliche Evidenz benutzen. Daraus wurde die schärfere Einsicht: **das Stage-Zünd-Problem ist ein Look-Ahead-Problem unter Unsicherheit**, weil weder ein absolutes noch ein relatives Kriterium eine tote Zwischenstufe von einem erreichten Fehlerboden unterscheiden kann.

Der billige Look-Ahead im Ableitungsraum funktioniert — nachdem eine Zwischenrunde gezeigt hatte, dass er scheinbar scheitert, in Wahrheit aber die Ableitungsschätzung das Signal überdeckte. Er wurde vom Offline-Klassifikator zum Suchmechanismus, weil sich zeigte, dass das Gate suchunabhängig ist und keine Rollback-Maschinerie braucht. Sein Geltungsbereich ist vermessen statt vermutet: sicher, wo die Ableitung die Struktur auflöst; unsicher, wo nicht; und beides vorab am Rauschboden ablesbar.

Und dann die Wendung. Der Cap beseitigt den Overshoot vollständig — bei bit-identischem Loss, also war die Eskalation reine Verschwendung. Aber die entdeckte Struktur bleibt exakt dieselbe wie drei Methodengenerationen zuvor. **Die Stage-Eskalation war ein Symptom, nicht die Ursache.** Was seit Gate 1 als das zu lösende Problem galt, war die sichtbare Begleiterscheinung eines anderen: die Suche findet die richtige Struktur auch dann nicht, wenn man ihr alle nötigen Terme hinlegt und alles andere verbietet.

Was das wert ist. Wissenschaftlich ist das ein besseres Ergebnis als ein Erfolg gewesen wäre. Das Projekt hat eine Hypothese über drei Generationen sauber verfolgt, mit vorab festgelegten Kriterien falsifiziert, den Mechanismus dennoch als eigenständigen Beitrag mit vermessenen Grenzen gerettet — und dabei die eigentliche Ursache freigelegt. Genau das ist die mechanistische Claim-C-Studie, die seit dem Pivot vom 2026-05-17 das Ziel war: nicht zu zeigen, dass EvoGrow gewinnt, sondern **wo gestuftes Wachstum hilft, wo es versagt und warum.**

Der nächste Schritt ist deshalb nicht die nächste Methodengeneration, sondern das Absichern des Belegs: v3 ungecappt auf System 31 zur Entkonfundierung, die Gitter-Entscheidung, die Systemklassifikation. Die offene Forschungsfrage — Suchkraft innerhalb einer Stufe — ist notiert und bleibt draußen.